Problem Statement:

To Analyze the data and generating insights that could help Netflix in deciding which type of shows/movies to produce and how they can grow the business in different countries

also To maintain its competitive edge and continue growing, Netflix needs actionable insights to determine what type of content to produce, which markets to focus on, and when to release new content.

Objective:

Understand the content distribution: Compare movies vs. TV shows, and analyze genres, ratings, and country-wise patterns.

Identify trends and patterns: Explore how content production has evolved over time and detect popular genres and regions.

Determine optimal release strategies: Find the best time (month/week) to launch new movies and TV shows.

Analyze key contributors: Highlight the most frequent actors, directors, and genres associated with Netflix content.

Provide data-driven recommendations: Suggest actionable steps for content production and market expansion to enhance engagement and subscriptions.

1.Import Libraries

In [82]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

2.Load Dataset

In [173…
```
df = pd.read_csv('netflix.csv')
df.head(2)
```

Out[173…

	show_id	type	title	director	cast	country	date_added	release_yea
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	202
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	202

3.Basic Data Analysis

shape of dataset

In [72]:
```
print("Shape of dataset:", df.shape)
```

```
Shape of dataset: (8807, 12)
```

All columns present in dataset

In [180…
```
df.dtypes
df.columns
```

Out[180…
```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_
added',
       'release_year', 'rating', 'duration', 'listed_in', 'descriptio
n'],
      dtype='object')
```

Data types

Information about Data Frame

shows Index range, Column names, Non-null count per column, Data types (dtypes), Memory usage

In [75]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
None
```

describes summary statistics, but only for numeric columns by default.

Includes: count, mean, std, min, 25%, 50% (median), 75%, max

In [76]:
```
df.describe(include='all').T
```

Out[76]:

	count	unique	top	freq	mean	std	min	
show_id	8807	8807	s8807	1	NaN	NaN	NaN	
type	8807	2	Movie	6131	NaN	NaN	NaN	
title	8807	8807	Zubaan	1	NaN	NaN	NaN	
director	6173	4528	Rajiv Chilaka	19	NaN	NaN	NaN	
cast	7982	7692	David Attenborough	19	NaN	NaN	NaN	
country	7976	748	United States	2818	NaN	NaN	NaN	
date_added	8797	1767	January 1, 2020	109	NaN	NaN	NaN	
release_year	8807.0	NaN	NaN	NaN	2014.180198	8.819312	1925.0	2
rating	8803	17	TV-MA	3207	NaN	NaN	NaN	
duration	8804	220	1 Season	1793	NaN	NaN	NaN	
listed_in	8807	514	Dramas, International Movies	362	NaN	NaN	NaN	
description	8807	8775	Paranormal activity at a lush, abandoned prope...	4	NaN	NaN	NaN	

4.Value counts and unique attributes

for unique valued columns

In [86]:
```
categorical_cols = [
    "type",
    "title",
    "rating"
]

for col in categorical_cols:
    print(f"Value counts for {col}:")
    print(df[col].value_counts())
    print("\nNumber of unique values:", df[col].nunique())
    print("Unique values:", df[col].unique())
    print("\n" + "-"*40 + "\n")
```

```
Value counts for type:
type
Movie      6131
TV Show    2676
Name: count, dtype: int64

Number of unique values: 2
Unique values: ['Movie' 'TV Show']

----------------------------------------

Value counts for title:
title
Zubaan                                 1
Dick Johnson Is Dead                   1
Blood & Water                          1
Ganglands                              1
Jailbirds New Orleans                  1
                                      ..
Bangkok Breaking                       1
Vendetta: Truth, Lies and The Mafia    1
The Starling                           1
The Great British Baking Show          1
Sankofa                                1
Name: count, Length: 8807, dtype: int64

Number of unique values: 8807
Unique values: ['Dick Johnson Is Dead' 'Blood & Water' 'Ganglands' ... 'Zo
mbieland'
 'Zoom' 'Zubaan']

----------------------------------------

Value counts for rating:
rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
TV-Y7-FV       6
NC-17          3
UR             3
74 min         1
84 min         1
66 min         1
Name: count, dtype: int64

Number of unique values: 17
Unique values: ['PG-13' 'TV-MA' 'PG' 'TV-14' 'TV-PG' 'TV-Y' 'TV-Y7' 'R' 'T
V-G' 'G'
 'NC-17' '74 min' '84 min' '66 min' 'NR' nan 'TV-Y7-FV' 'UR']
```

5.Missing values and data cleaning

In [24]:
```
missing = df.isnull().sum()
print(missing)
```

```
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
```

Filling the missing values

In [87]:
```
df = df.fillna({
    'director': 'Unknown Director',
    'cast': 'Unknown Cast',
    'country': 'Unknown Country',
    'rating': 'Unknown Rating',
    'duration':'0'

})
df.head(2)
```

Out[87]:

	show_id	type	title	director	cast	country	date_added	release_ye
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	Unknown Cast	United States	September 25, 2021	20:
1	s2	TV Show	Blood & Water	Unknown Director	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	20

6.conversion of categorical attributes to 'category'

In [94]:
```
categorical_cols = ['type', 'director', 'cast', 'country', 'rating', 'lis

# Convert to category dtype
for col in categorical_cols:
    if col in df.columns:  # ensure column exists
```

```
        df[col] = df[col].astype('category')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   object
 1   type          8807 non-null   category
 2   title         8807 non-null   object
 3   director      8807 non-null   category
 4   cast          8807 non-null   category
 5   country       8807 non-null   category
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64
 8   rating        8807 non-null   category
 9   duration      8807 non-null   object
 10  listed_in     8807 non-null   category
 11  description   8807 non-null   object
dtypes: category(6), int64(1), object(5)
memory usage: 1.0+ MB
```

7.Un-nesting Multivalued Columns

In [176…

```
def unnest_multivalued_columns(df, columns, sep=','):
    df = df.copy()
    if isinstance(columns, str):
        columns = [columns]
    for col in columns:
        df[col] = df[col].apply(lambda x: [i.strip() for i in str(x).spli
    for col in columns:
        df = df.explode(col, ignore_index=True)
    return df
```

In [177…

```
df_clean = unnest_multivalued_columns(df, ['country', 'listed_in', 'cast'
```

8.Missing Value & Outlier check

In [69]:

```
df = df.copy()
```

In [70]:

```
mv_counts = df.isna().sum().sort_values(ascending=False)
mv_pct = (df.isna().mean()*100).sort_values(ascending=False)

missing_summary = pd.concat([mv_counts, mv_pct], axis=1)
missing_summary.columns = ["Missing Count", "Missing %"]
print(missing_summary)
```

```
              Missing Count  Missing %
director               2634  29.908028
country                 831   9.435676
cast                    825   9.367549
date_added               10   0.113546
rating                    4   0.045418
duration                  3   0.034064
show_id                   0   0.000000
type                      0   0.000000
title                     0   0.000000
release_year              0   0.000000
listed_in                 0   0.000000
description               0   0.000000
```

In [71]:
```
# Extract number from "90 min" or "2 Seasons"
df["duration_num"] = df["duration"].str.extract(r"(\d+)").astype(float)

# Keep separate views for valid numeric use-cases
movies = df[df["type"] == "Movie"].copy()          # minutes
shows  = df[df["type"] == "TV Show"].copy()        # seasons
```

In [77]:
```
def iqr_outliers(s: pd.Series):
    s = s.dropna()
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    lo = q1 - 1.5*iqr
    hi = q3 + 1.5*iqr
    mask = (s < lo) | (s > hi)
    return {
        "Q1": q1, "Q3": q3, "IQR": iqr,
        "Lower Bound": lo, "Upper Bound": hi,
        "Outlier Count": int(mask.sum())
    }, mask
```

Outliers for release years

In [178…
```
year_stats, year_mask = iqr_outliers(df["release_year"])
print("Release Year IQR summary:", year_stats)

# Inspect a few old/new outliers
df_year_outliers = df.loc[year_mask, ["title", "type", "release_year", "d
df_year_outliers.head(5)
```

```
Release Year IQR summary: {'Q1': 2013.0, 'Q3': 2019.0, 'IQR': 6.0, 'Lower
Bound': 2004.0, 'Upper Bound': 2028.0, 'Outlier Count': 719}
```

Out[178…

	title	type	release_year	date_added
7	Sankofa	Movie	1993	September 24, 2021
22	Avvai Shanmughi	Movie	1996	September 21, 2021
24	Jeans	Movie	1998	September 21, 2021
26	Minsara Kanavu	Movie	1997	September 21, 2021
41	Jaws	Movie	1975	September 16, 2021

unnesting single column to avoid duplicate data

In [37]:
```
df_m = df[df['type'] == 'Movie']
df_t = df[df['type'] == 'TV Show']
```

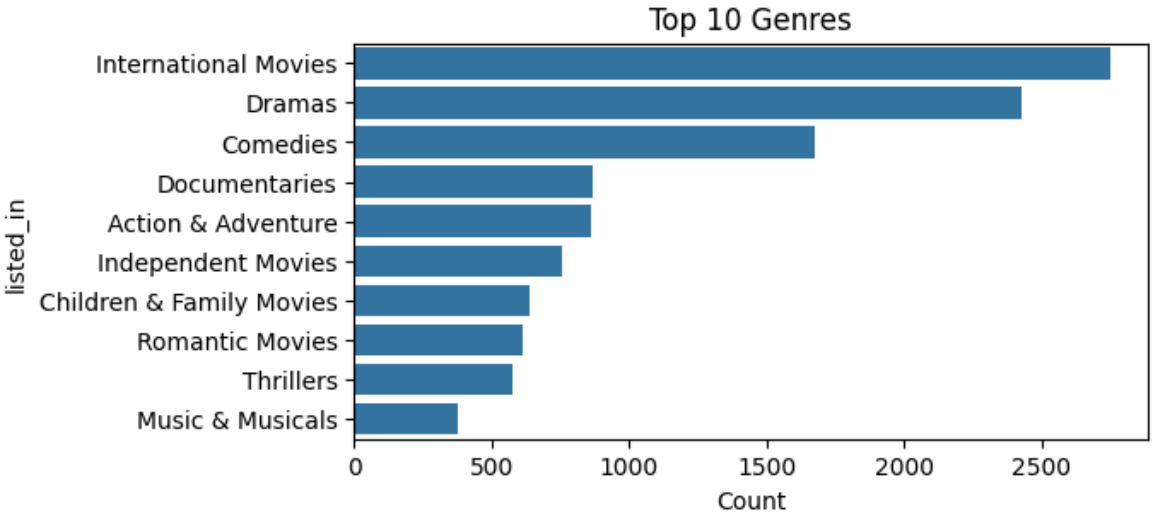
In [38]:
```
df_genrem = df_m.copy()
df_genrem['listed_in'] = df_genrem['listed_in'].astype(str).str.split(',
df_genrem = df_genrem.explode('listed_in')

df_countrym = df_m.copy()
df_countrym['country'] = df_countrym['country'].astype(str).str.split(',
df_countrym = df_countrym.explode('country')

df_castm = df_m.copy()
df_castm['cast'] = df_castm['cast'].astype(str).str.split(', ')
df_castm = df_castm.explode('cast')

df_directorm = df_m.copy()
df_directorm['director'] = df_directorm['director'].astype(str).str.split
df_directorm = df_directorm.explode('director')
df_directorm = df_directorm[df_directorm['director'] != 'Unknown Director
```

In [39]:
```
df_genret = df_t.copy()
df_genret['listed_in'] = df_genret['listed_in'].astype(str).str.split(',
df_genret = df_genret.explode('listed_in')

df_countryt = df_t.copy()
df_countryt['country'] = df_countryt['country'].astype(str).str.split(',
df_countryt = df_countryt.explode('country')

df_castt = df_t.copy()
df_castt['cast'] = df_castt['cast'].astype(str).str.split(', ')
df_castt = df_castt.explode('cast')

df_directort = df_t.copy()
df_directort['director'] = df_directort['director'].astype(str).str.split
df_directort = df_directort.explode('director')
df_directort = df_directort[df_directort['director'] != 'Unknown Director
```

univariate analysis (unnested data)

Top genres for movies

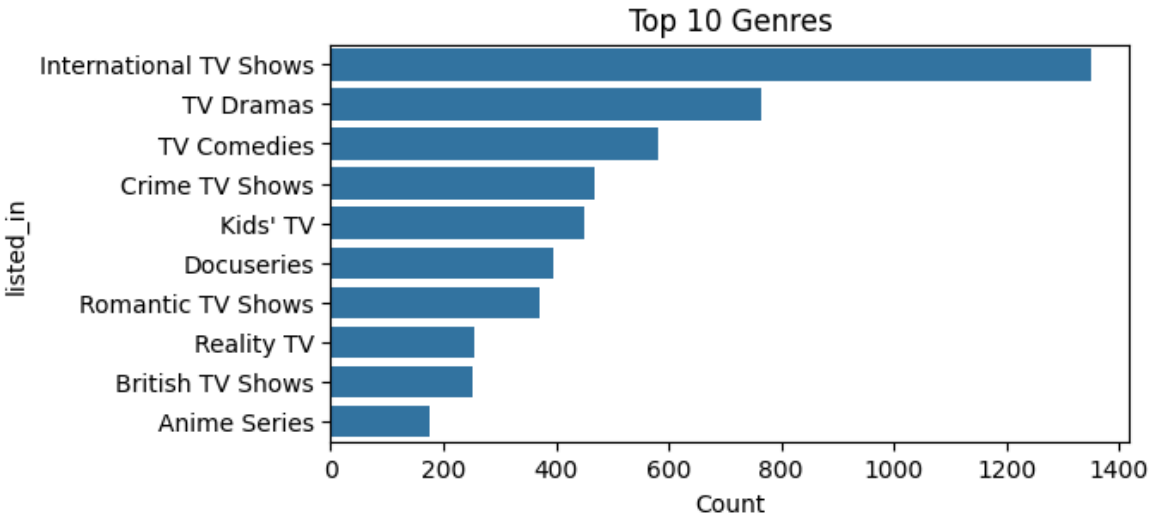
In [138…
```
plt.figure(figsize=(6,3))
genre_counts = df_genrem['listed_in'].value_counts()
sns.barplot(y=genre_counts.head(10).index, x=genre_counts.head(10).values
plt.title("Top 10 Genres")
plt.xlabel("Count")
plt.show()
```



Top genres for TvShows

In [137…

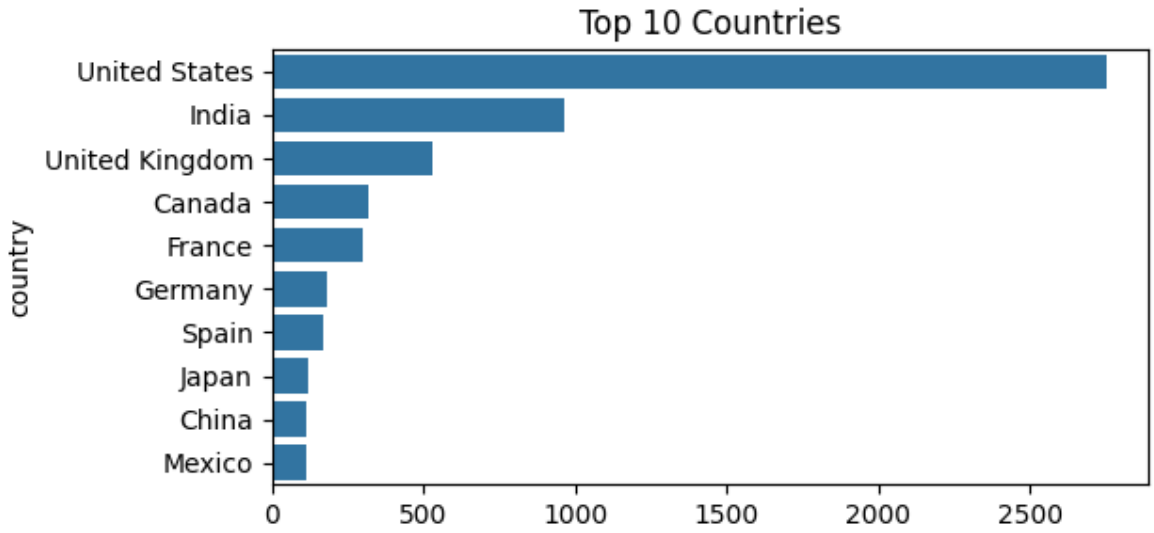
```
plt.figure(figsize=(6,3))
genre_counts = df_genret['listed_in'].value_counts()
sns.barplot(y=genre_counts.head(10).index, x=genre_counts.head(10).values
plt.title("Top 10 Genres")
plt.xlabel("Count")
plt.show()
```



Top Countries with Most Content for movies

In [136…

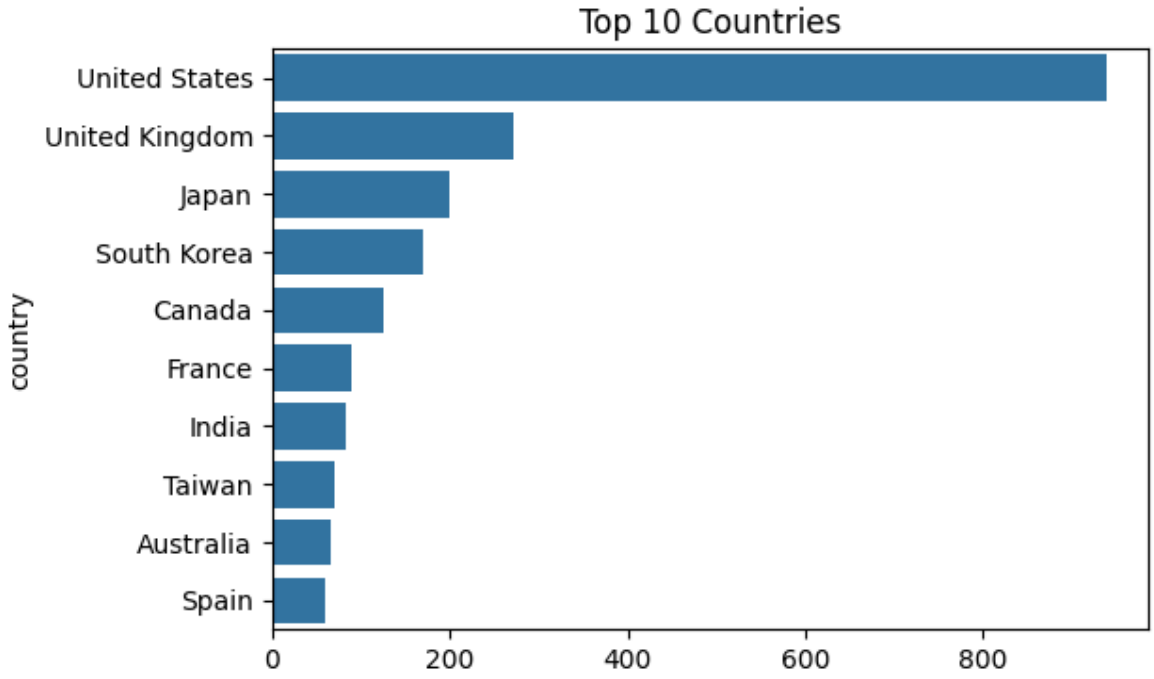
```
plt.figure(figsize=(6,3))
df_country_filteredm = df_countrym[~df_countrym['country'].isin(['Unknown
country_counts = df_country_filteredm['country'].value_counts()
sns.barplot(y=country_counts.head(10).index, x=country_counts.head(10).va
plt.title("Top 10 Countries")
plt.show()
```



Top Countries with Most Content for TvSHows

In [135…

```
plt.figure(figsize=(6,4))
df_country_filteredt = df_countryt[~df_countryt['country'].isin(['Unknown
country_counts = df_country_filteredt['country'].value_counts()
sns.barplot(y=country_counts.head(10).index, x=country_counts.head(10).va
plt.title("Top 10 Countries")
plt.show()
```

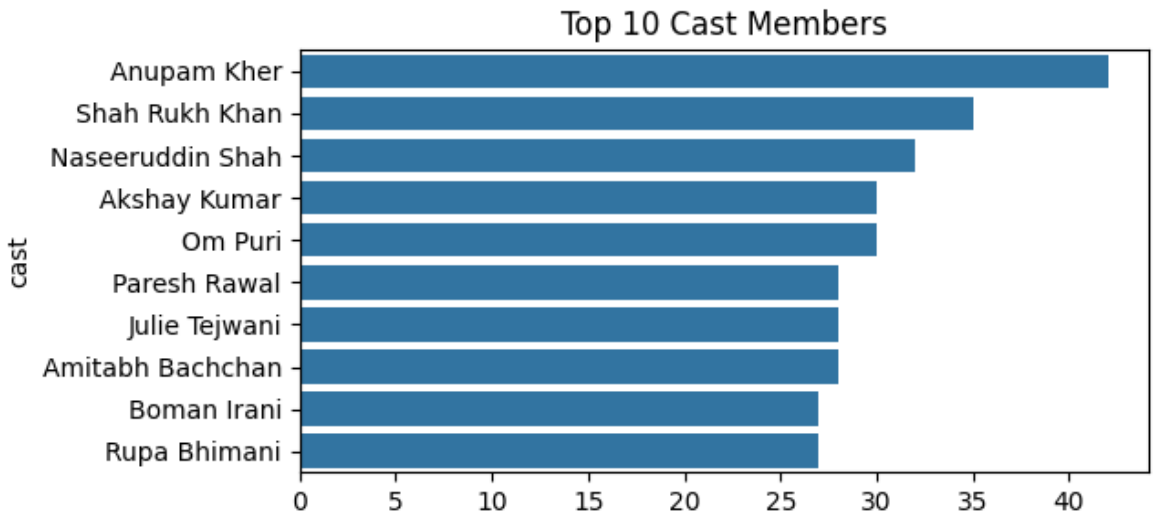


Top Cast Members by Number of Appearances for Movies

In [139…

```
plt.figure(figsize=(6,3))
df_cast_filtered = df_castm[~df_castm['cast'].isin(['Unknown Cast', 'Othe
cast_counts = df_cast_filtered['cast'].value_counts()
sns.barplot(y=cast_counts.head(10).index, x=cast_counts.head(10).values)
```

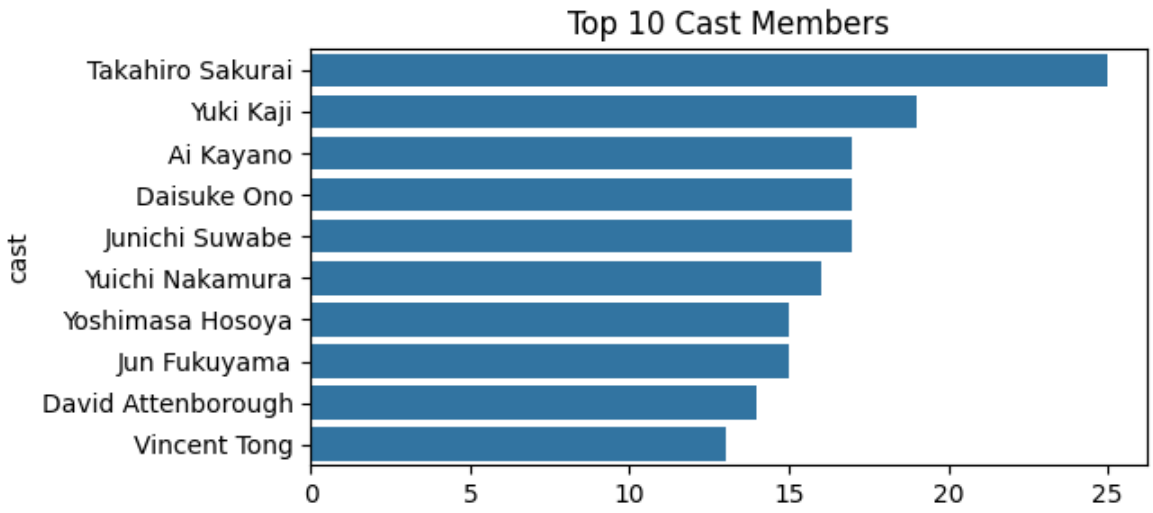
```
plt.title("Top 10 Cast Members")
plt.show()
```



Top Cast Members by Number of Appearances for TvShows

In [140…

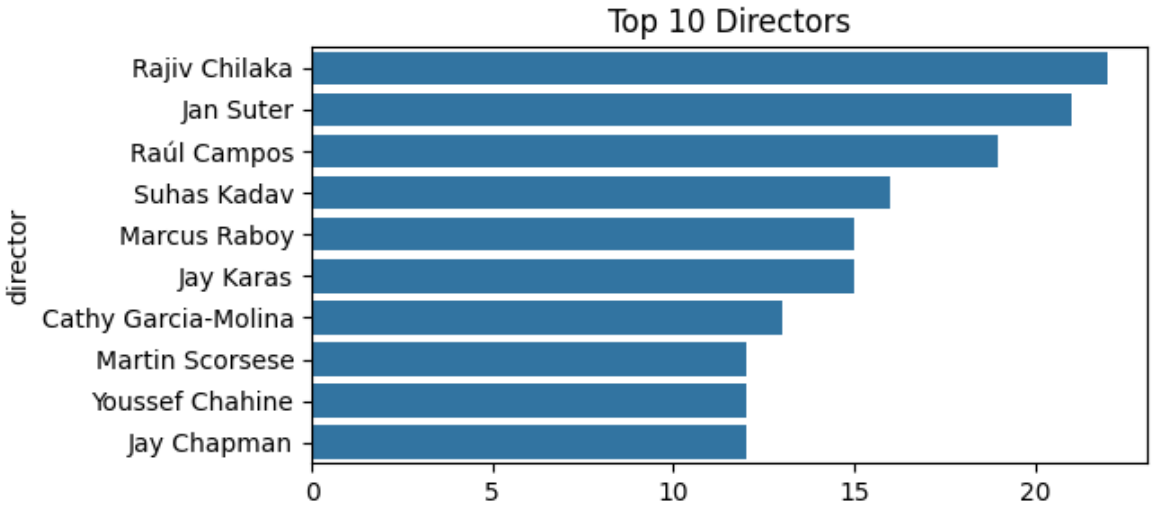
```
plt.figure(figsize=(6,3))
df_cast_filtered = df_castt[~df_castt['cast'].isin(['Unknown Cast', 'Othe
cast_counts = df_cast_filtered['cast'].value_counts()
sns.barplot(y=cast_counts.head(10).index, x=cast_counts.head(10).values)
plt.title("Top 10 Cast Members")
plt.show()
```



Top Directors by Number of Titles for Movies

In [141…

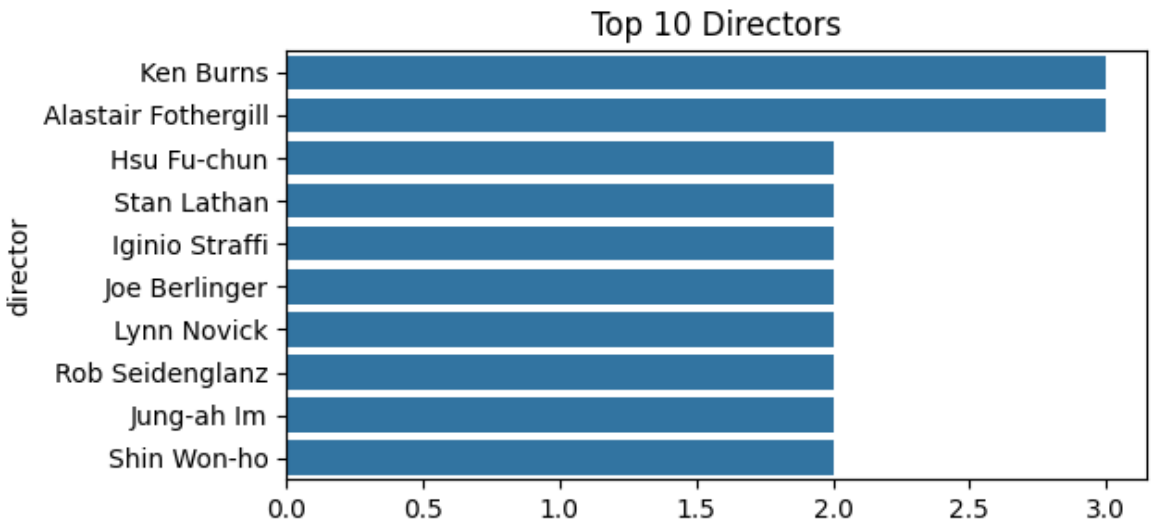
```
plt.figure(figsize=(6,3))
df_director_filtered = df_directorm[~df_directorm['director'].isin(['Unkn
director_counts = df_director_filtered['director'].value_counts()
sns.barplot(y=director_counts.head(10).index, x=director_counts.head(10).
plt.title("Top 10 Directors")
plt.show()
```



Top Directors by Number of Titles for TvShows

In [142…

```
plt.figure(figsize=(6,3))
df_director_filtered = df_directort[~df_directort['director'].isin(['Unkn
director_counts = df_director_filtered['director'].value_counts()
sns.barplot(y=director_counts.head(10).index, x=director_counts.head(10).
plt.title("Top 10 Directors")
plt.show()
```

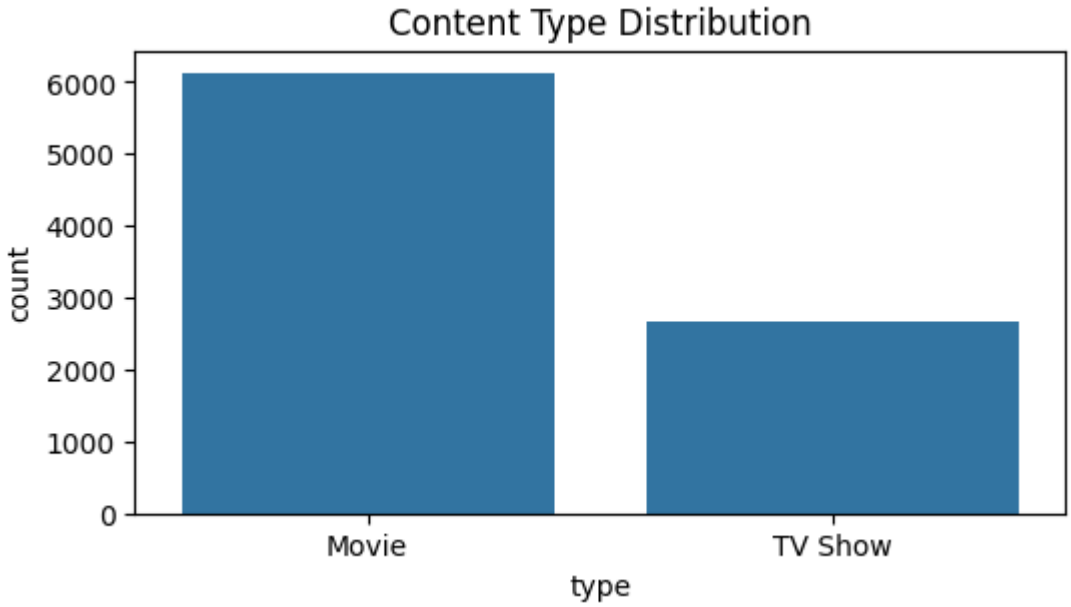


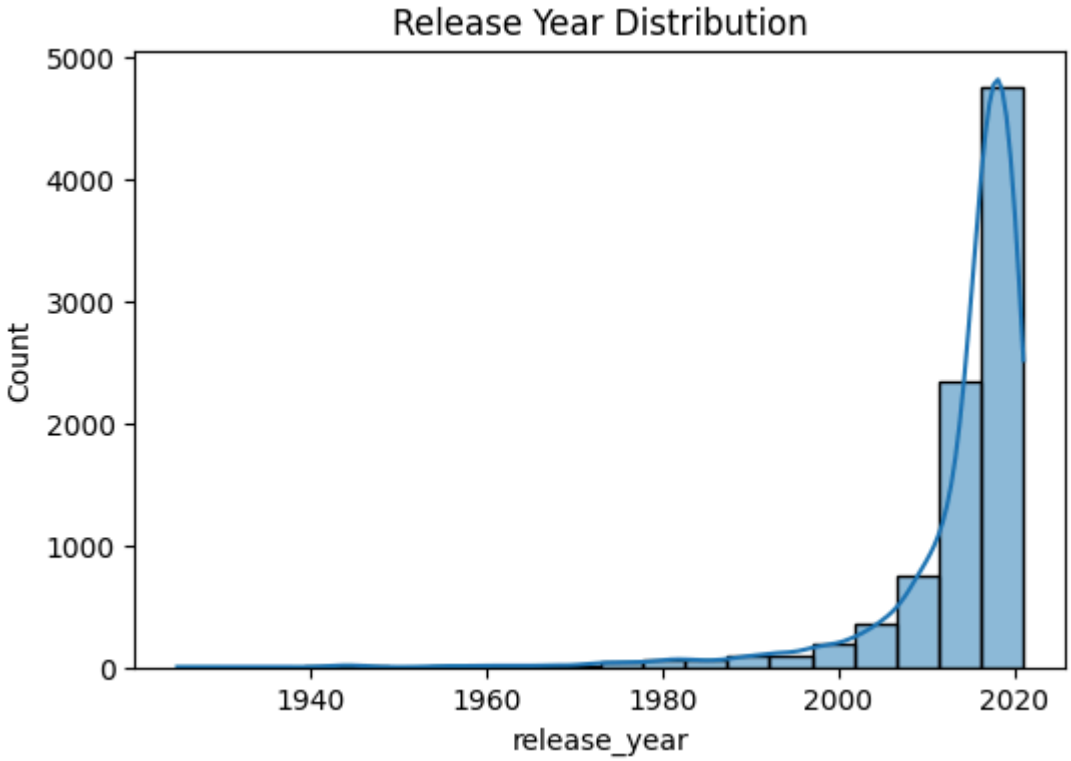
Univariate Analysis for original data

In [143…

```
plt.figure(figsize=(6,3))
print(df['type'].value_counts())
print((df['type'].value_counts(normalize=True)*100).apply(lambda x: f"{x:
sns.countplot(x='type', data=df)
plt.title("Content Type Distribution")
plt.show()

# Release year distribution
plt.figure(figsize=(6,4))
sns.histplot(df['release_year'], bins=20, kde=True)
plt.title("Release Year Distribution")
plt.show()
```

```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
type
Movie      69.62%
TV Show    30.38%
Name: proportion, dtype: object
```



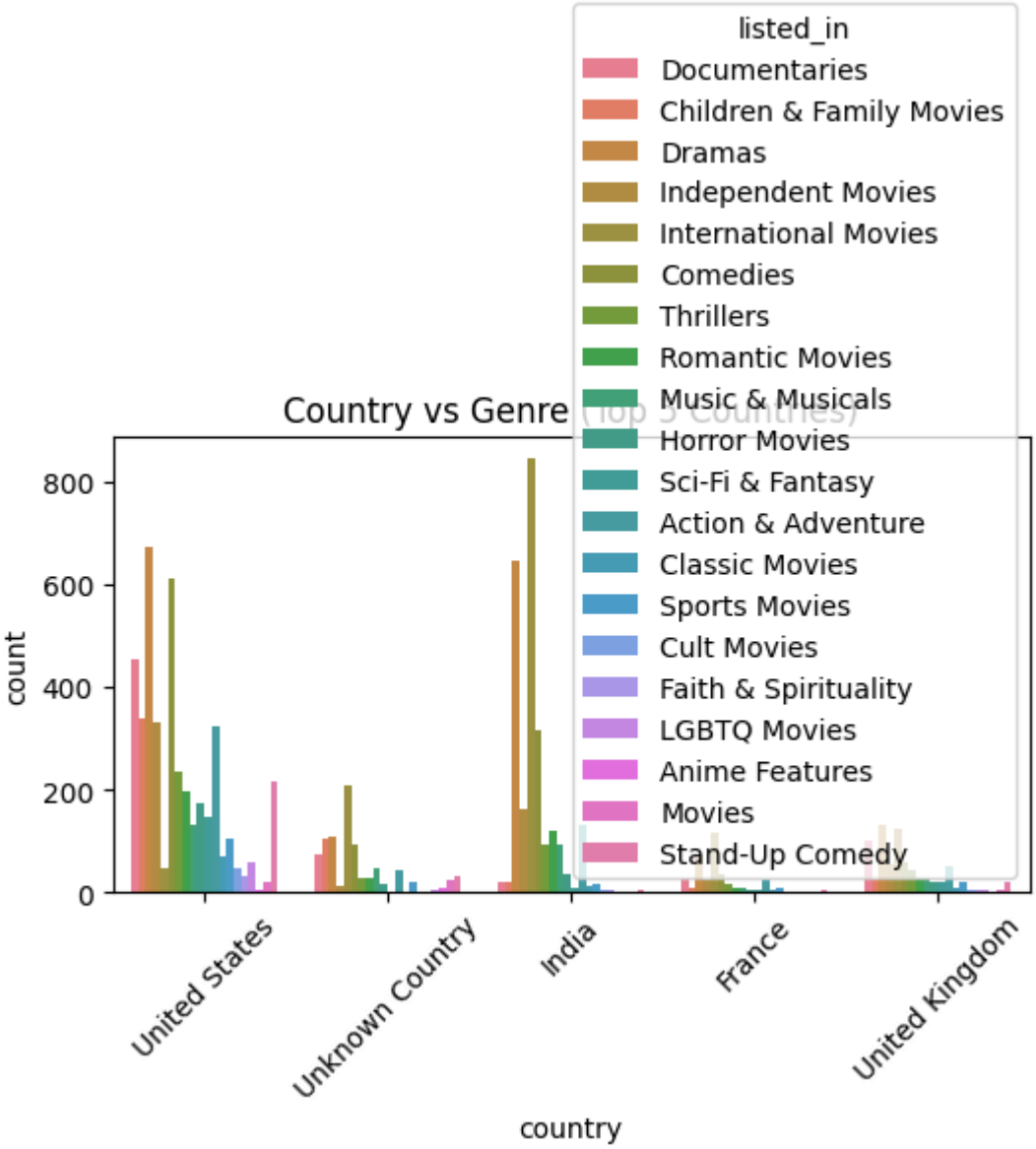


Bivariate Analysis

country vs genre

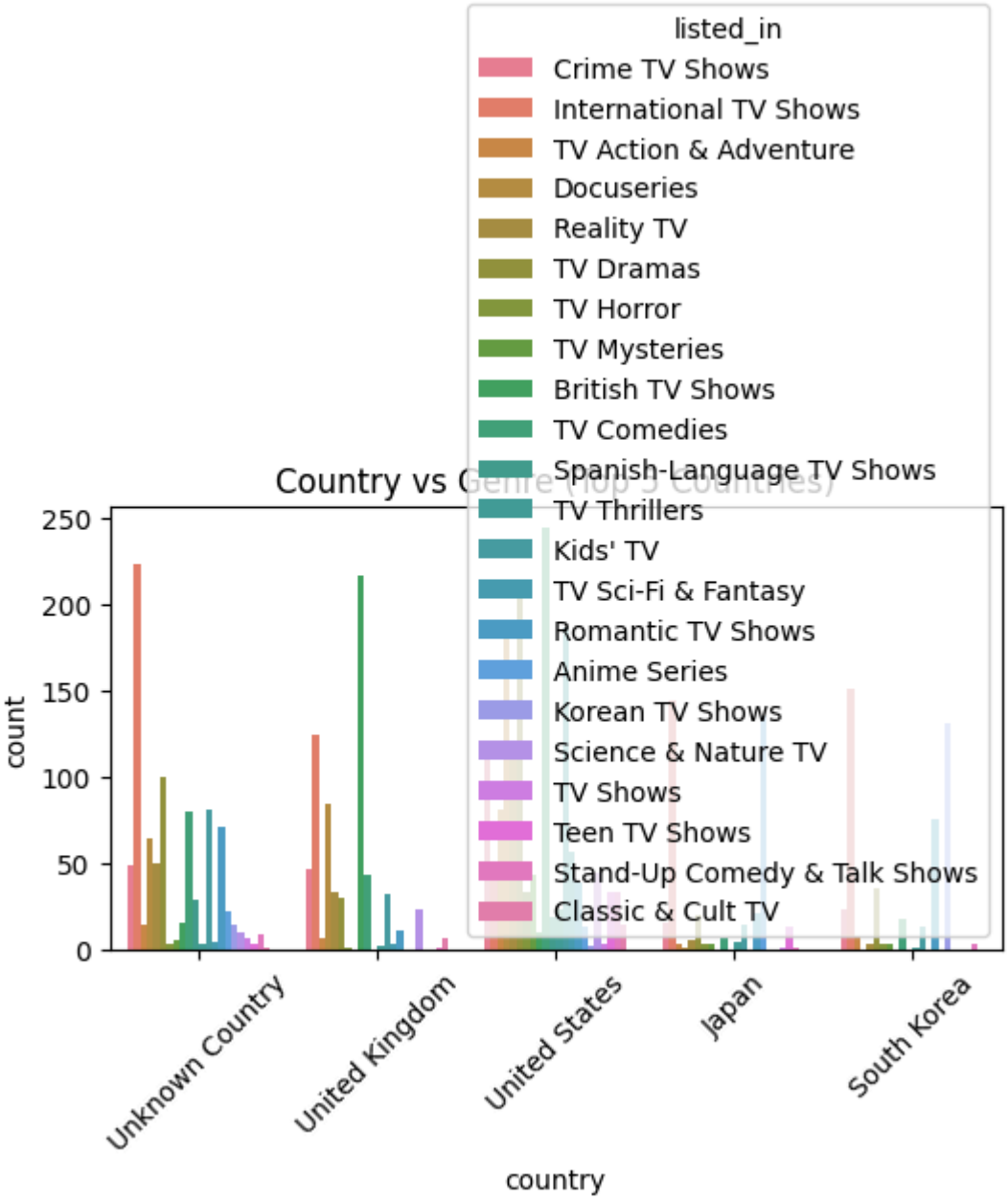
In [53]:
```
df_m = df[df['type'] == 'Movie']
df_t = df[df['type'] == 'TV Show']
```

In [171…
```
#Country vs Genre
df_country_genrem = df_m.copy()
df_country_genrem['country'] = df_country_genrem['country'].astype(str).s
df_country_genrem['listed_in'] = df_country_genrem['listed_in'].astype(st
df_country_genrem = df_country_genrem.explode('country').explode('listed_
df_country_genrem = df_country_genrem.drop_duplicates(['show_id', 'listed

top_countries = df_country_genrem['country'].value_counts().head(5).index
plt.figure(figsize=(6,3))
sns.countplot(data=df_country_genrem[df_country_genrem['country'].isin(to
              x='country', hue='listed_in')
plt.title("Country vs Genre (Top 5 Countries)")
plt.xticks(rotation=45)
plt.show()
```



In [172…
```
df_country_genret = df_t.copy()
df_country_genret['country'] = df_country_genret['country'].astype(str).s
df_country_genret['listed_in'] = df_country_genret['listed_in'].astype(st
df_country_genret = df_country_genret.explode('country').explode('listed_
df_country_genret = df_country_genret.drop_duplicates(['show_id', 'listed
```

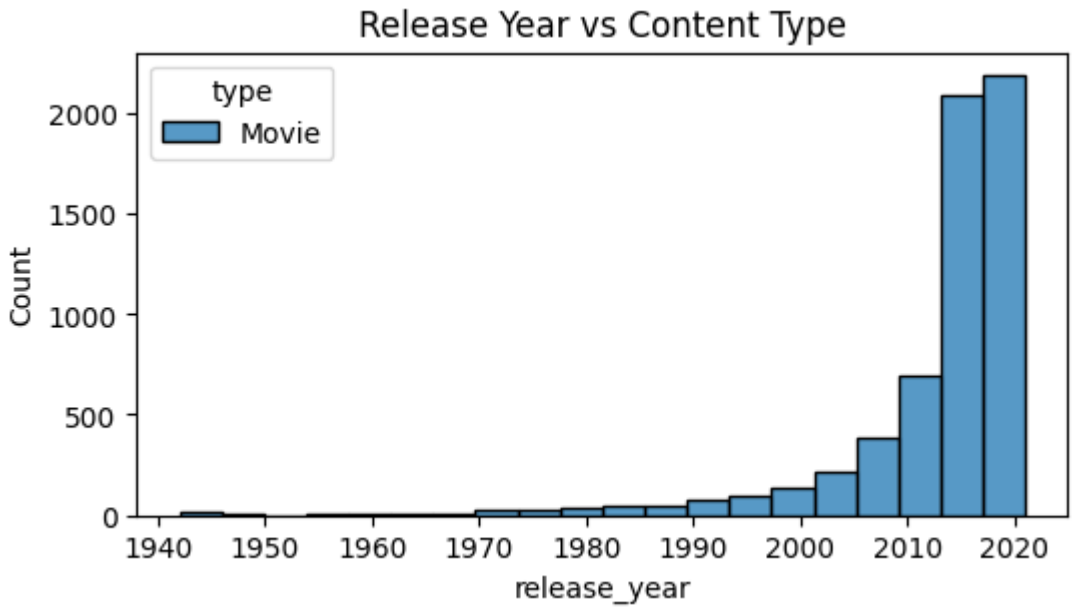
```
top_countries = df_country_genret['country'].value_counts().head(5).index
plt.figure(figsize=(6,3))
sns.countplot(data=df_country_genret[df_country_genret['country'].isin(to
              x='country', hue='listed_in')
plt.title("Country vs Genre (Top 5 Countries)")
plt.xticks(rotation=45)
plt.show()
```



Release Year vs Type

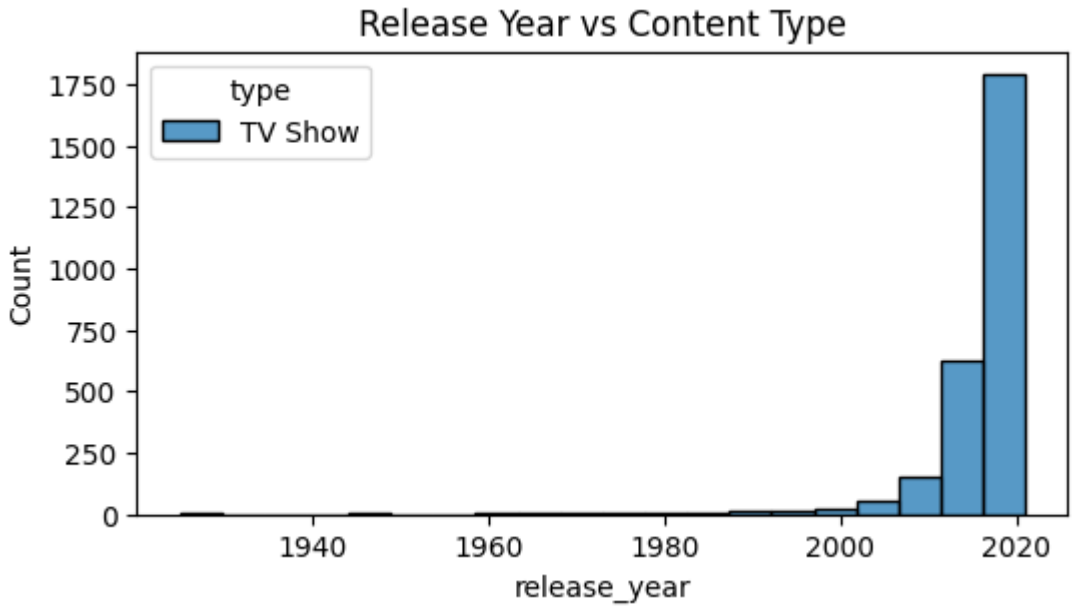
In [146…

```
#Release Year vs Type
plt.figure(figsize=(6,3))
sns.histplot(data=df_m, x='release_year', hue='type', multiple='stack', b
plt.title("Release Year vs Content Type")
plt.show()
```



In [147…

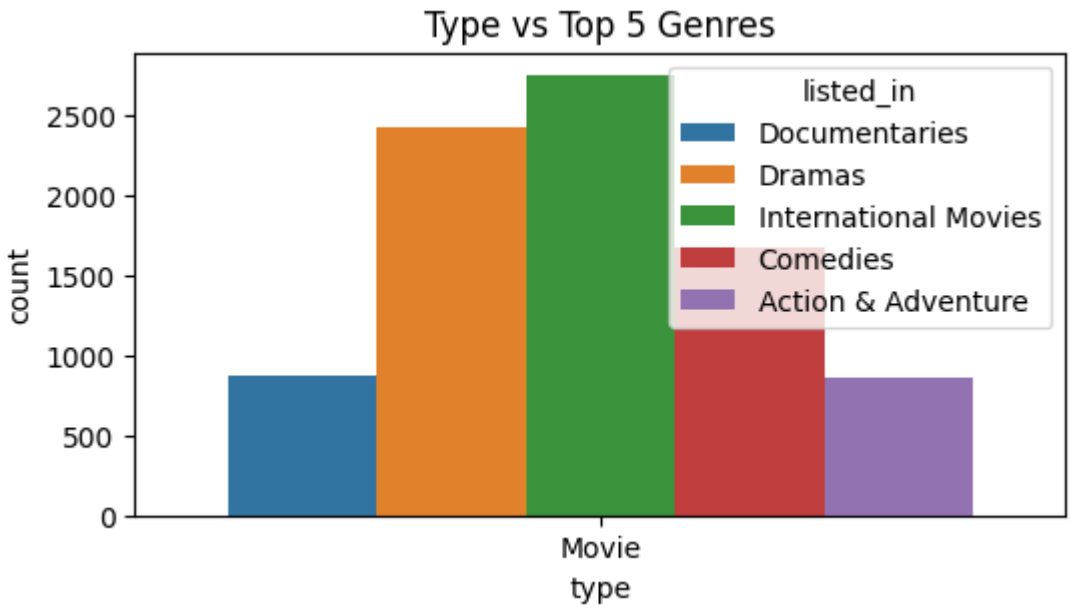
```
plt.figure(figsize=(6,3))
sns.histplot(data=df_t, x='release_year', hue='type', multiple='stack', b
plt.title("Release Year vs Content Type")
plt.show()
```



Type vs Genre

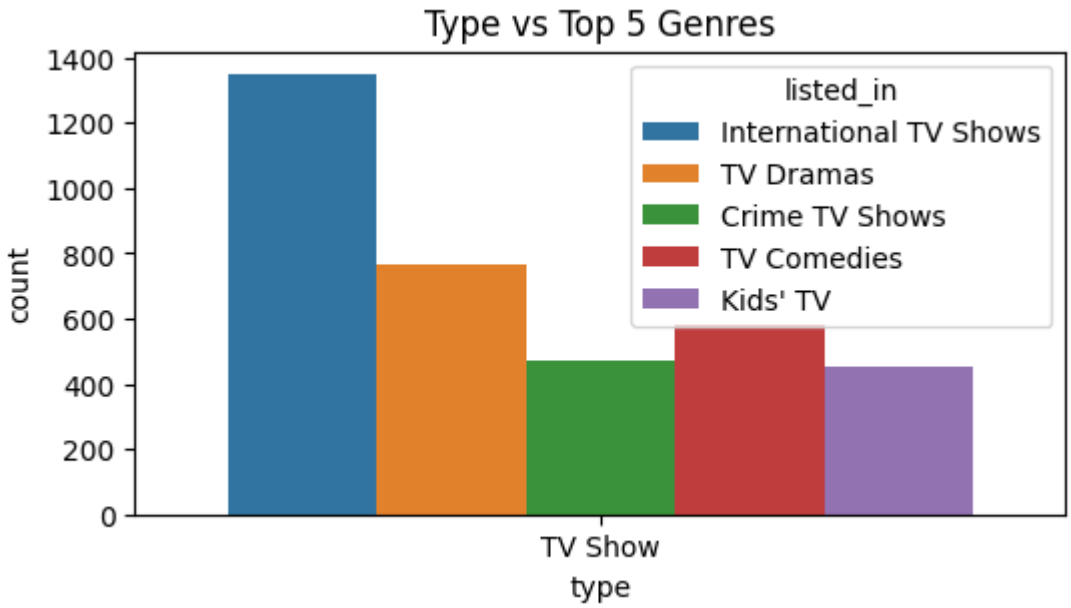
In [148…

```
#Bivariate Analysis (Avoiding Duplication)
#Type vs Genre
df_type_genrem = df_m.copy()
df_type_genrem['listed_in'] = df_type_genrem['listed_in'].astype(str).str
df_type_genrem = df_type_genrem.explode('listed_in').drop_duplicates(['sh

top_genres = df_type_genrem['listed_in'].value_counts().head(5).index
plt.figure(figsize=(6,3))
sns.countplot(data=df_type_genrem[df_type_genrem['listed_in'].isin(top_ge
              x='type', hue='listed_in')
plt.title("Type vs Top 5 Genres")
plt.show()
```



In [149…

```
df_type_genret = df_t.copy()
df_type_genret['listed_in'] = df_type_genret['listed_in'].astype(str).str
df_type_genret = df_type_genret.explode('listed_in').drop_duplicates(['sh

top_genres = df_type_genret['listed_in'].value_counts().head(5).index
plt.figure(figsize=(6,3))
sns.countplot(data=df_type_genret[df_type_genret['listed_in'].isin(top_ge
              x='type', hue='listed_in')
plt.title("Type vs Top 5 Genres")
plt.show()
```



BoxPlot

In [61]:

```
df_duration = df.copy()

# Extract numeric duration and unit
df_duration['duration_num'] = df_duration['duration'].str.extract('(\d+)'
df_duration['duration_unit'] = df_duration['duration'].str.extract('([a-z
```

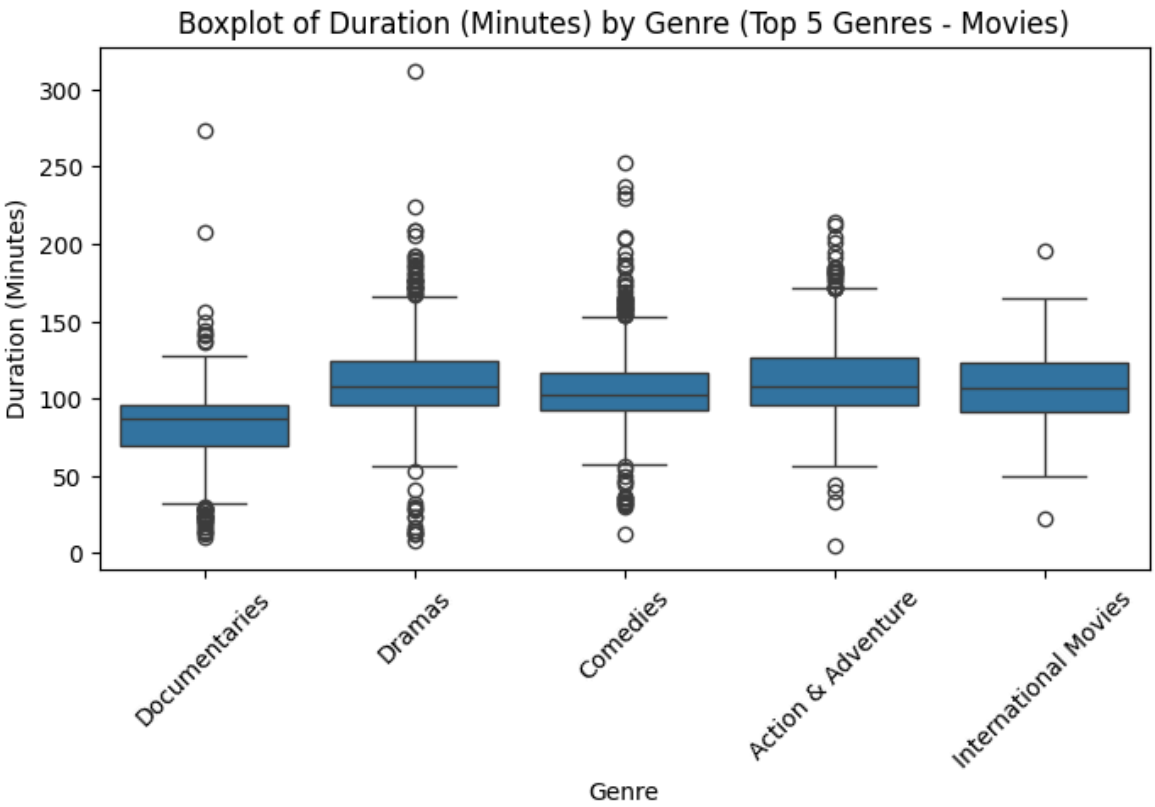
```
# Separate Movies and TV Shows
movies = df_duration[df_duration['type'] == 'Movie']
tv_shows = df_duration[df_duration['type'] == 'TV Show']
```

```
<>:4: SyntaxWarning: invalid escape sequence '\d'
<>:4: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipython-input-1481918209.py:4: SyntaxWarning: invalid escape sequence
'\d'
  df_duration['duration_num'] = df_duration['duration'].str.extract('(\d
+)').astype(float)
```

Duration (Minutes) by Genre (Movies – Top 5 Genres)

In [166…

```
# ---------- Boxplot 1: Genre vs Duration (Movies only) ----------
plt.figure(figsize=(8,4))
# Get top 5 most frequent genres
top_genres = movies['listed_in'].astype(str).str.split(', ').explode().va
movies_genre = movies.copy()
# Take the first listed genre
movies_genre['genre'] = movies_genre['listed_in'].astype(str).str.split('
sns.boxplot(x='genre', y='duration_num', data=movies_genre[movies_genre['
plt.title("Boxplot of Duration (Minutes) by Genre (Top 5 Genres - Movies)
plt.xlabel("Genre")
plt.ylabel("Duration (Minutes)")
plt.xticks(rotation=45)
plt.show()
```

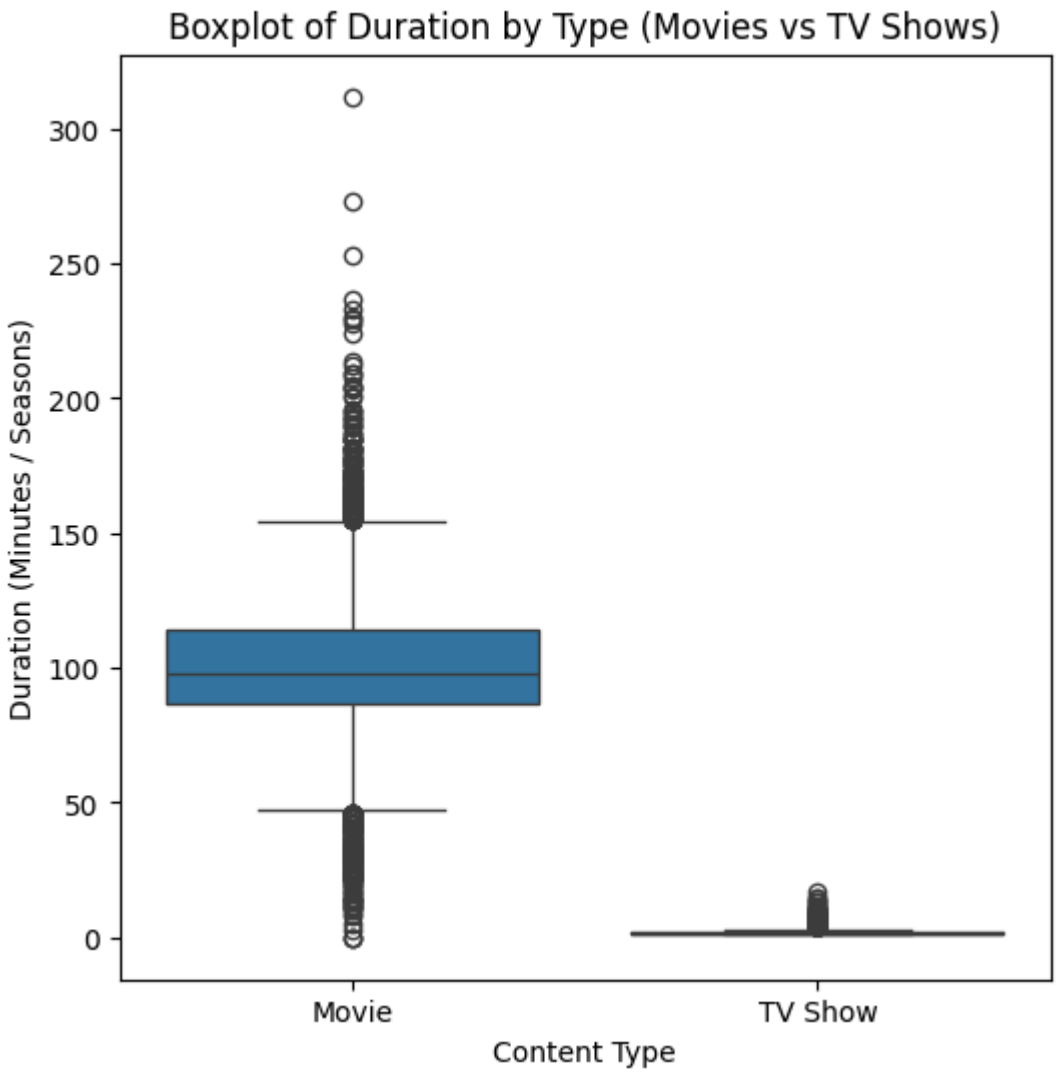


Duration by Type (Movies vs TV Shows)

In [167…

```
# ---------- Boxplot 2: Type vs Duration ----------
plt.figure(figsize=(6,6))
sns.boxplot(x='type', y='duration_num', data=df_duration)
plt.title("Boxplot of Duration by Type (Movies vs TV Shows)")
```

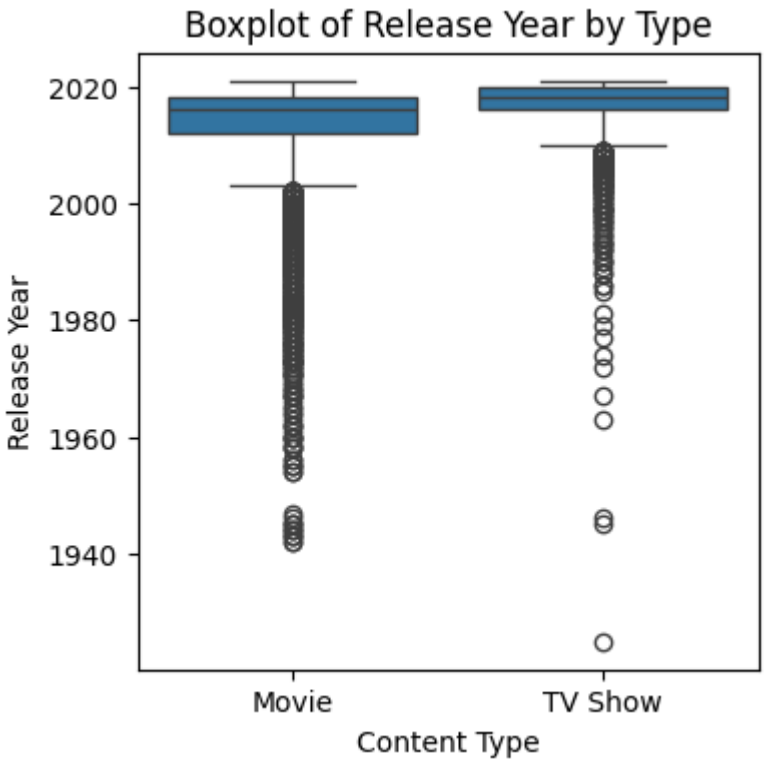
```
plt.xlabel("Content Type")
plt.ylabel("Duration (Minutes / Seasons)")
plt.show()
```



Release Year by Type (Movies vs TV Shows)

In [179…

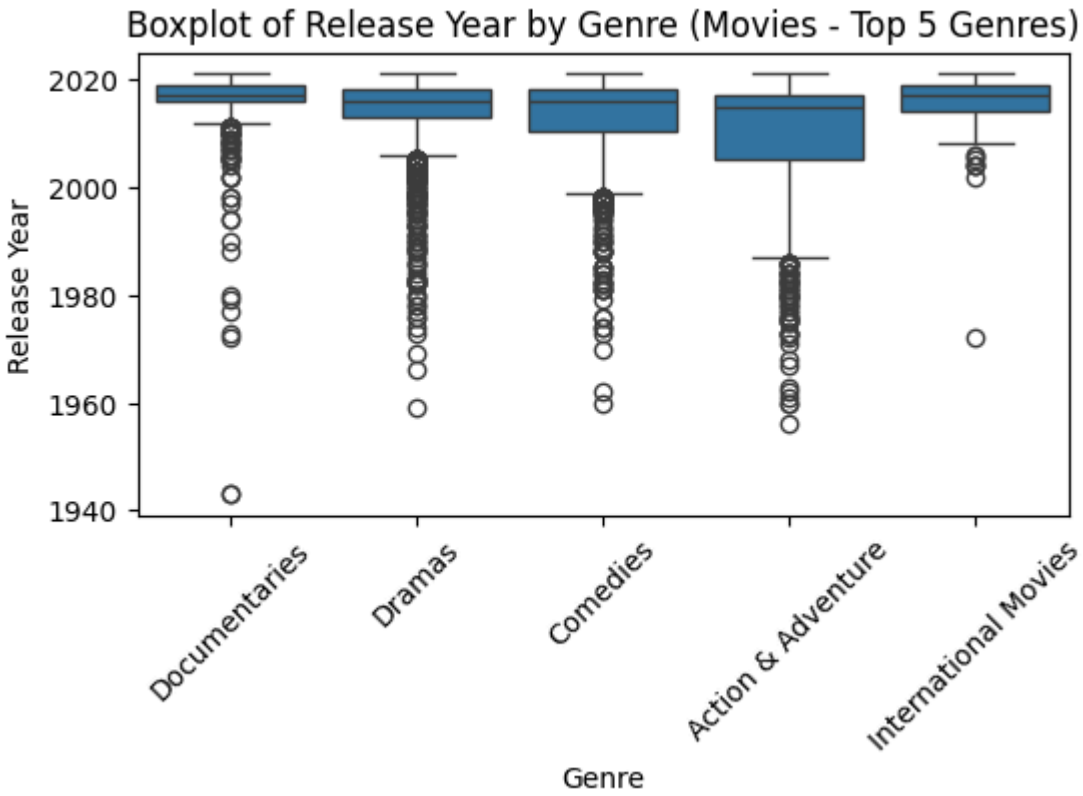
```
# ---------- Boxplot 3: Type vs Release Year ----------
plt.figure(figsize=(4,4))
sns.boxplot(x='type', y='release_year', data=df_duration)
plt.title("Boxplot of Release Year by Type")
plt.xlabel("Content Type")
plt.ylabel("Release Year")
plt.show()
```



Release Year by Genre (Movies – Top 5 Genres)

In [170…

```
# ---------- Boxplot 4: Genre vs Release Year (Movies only) ----------
plt.figure(figsize=(6,3))
sns.boxplot(x='genre', y='release_year', data=movies_genre[movies_genre['
plt.title("Boxplot of Release Year by Genre (Movies - Top 5 Genres)")
plt.xlabel("Genre")
plt.ylabel("Release Year")
plt.xticks(rotation=45)
plt.show()
```



number of movies produced in each country vs number of Tvshows produced in each country

In [121…
```
df_country = df.copy()
df_country['country'] = df_country['country'].astype(str).str.split(', ')
df_country = df_country.explode('country')
```

In [122…
```
df_movies = df_country[df_country['type'] == 'Movie']
df_tvshows = df_country[df_country['type'] == 'TV Show']
```

In [123…
```
movies_count = (
    df_movies.groupby('country')['title']
    .nunique()  # count unique titles
    .sort_values(ascending=False)
    .head(10)
)
print("Top 10 countries producing Movies:")
print(movies_count)
```

```
Top 10 countries producing Movies:
country
United States      2751
India               962
United Kingdom      532
Unknown Country     440
Canada              319
France              303
Germany             182
Spain               171
Japan               119
China               114
Name: title, dtype: int64
```

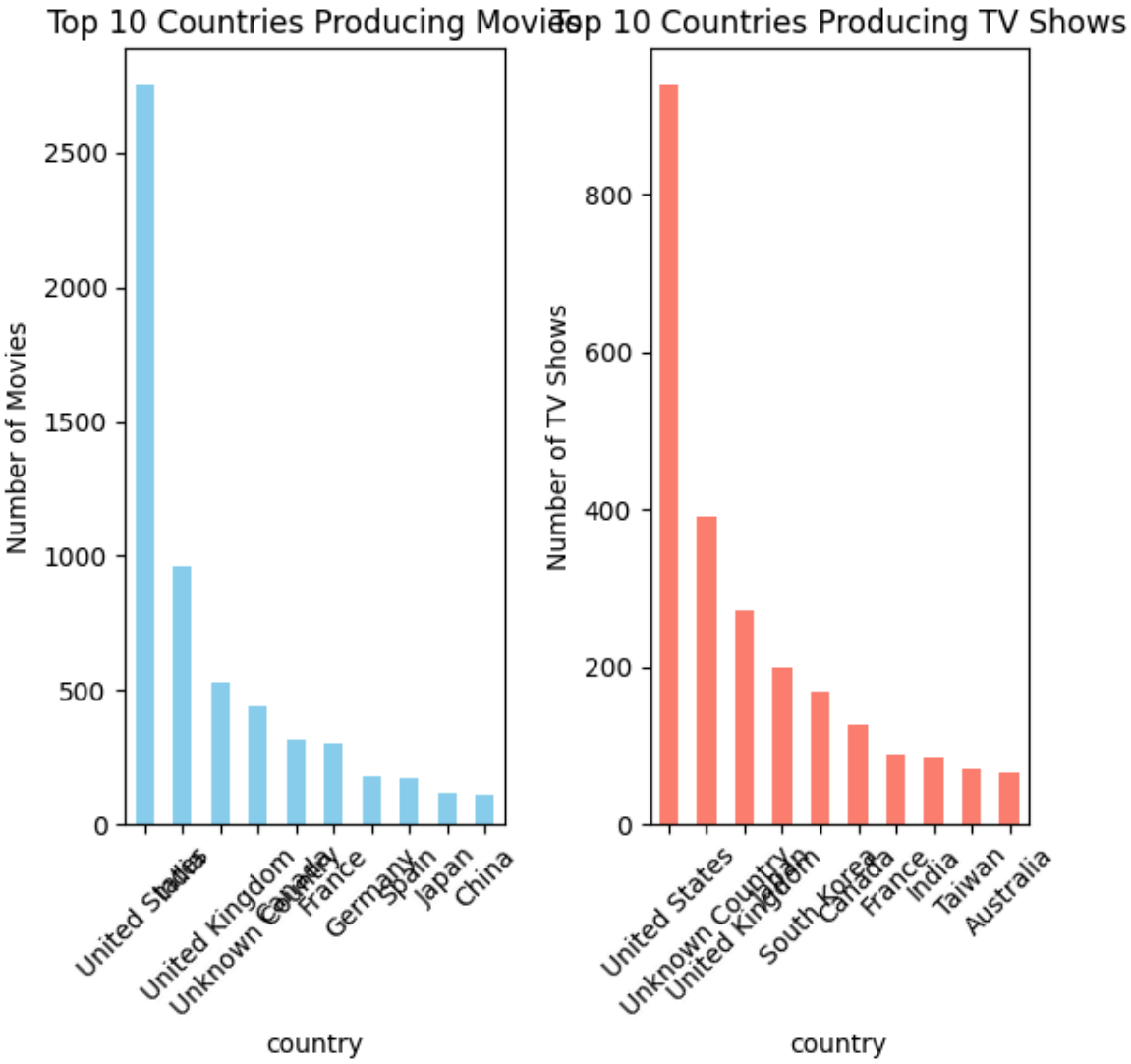
In [124…

```
tvshows_count = (
    df_tvshows.groupby('country')['title']
    .nunique()
    .sort_values(ascending=False)
    .head(10)
)
print("Top 10 countries producing TV Shows:")
print(tvshows_count)
```

```
Top 10 countries producing TV Shows:
country
United States      938
Unknown Country    391
United Kingdom     272
Japan              199
South Korea        170
Canada             126
France              90
India               84
Taiwan              70
Australia           66
Name: title, dtype: int64
```

In [181…

```
plt.figure(figsize=(6, 6))

# Movies
plt.subplot(1, 2, 1)
movies_count.plot(kind='bar', color='skyblue')
plt.title('Top 10 Countries Producing Movies')
plt.ylabel('Number of Movies')
plt.xticks(rotation=45)

# TV Shows
plt.subplot(1, 2, 2)
tvshows_count.plot(kind='bar', color='salmon')
plt.title('Top 10 Countries Producing TV Shows')
plt.ylabel('Number of TV Shows')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



best time to launch a TV show/Movie

In [111…
```
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
```

In [112…
```
df['week'] = df['date_added'].dt.isocalendar().week
df['month'] = df['date_added'].dt.month
```

In [113…
```
df_movies = df[df['type'] == 'Movie']
df_tvshows = df[df['type'] == 'TV Show']
```

In [114…
```
movies_week = (
    df_movies.groupby('week')['show_id']
    .count()
    .sort_values(ascending=False)
)
print("Best weeks for Movies:")
print(movies_week.head(5))
```

```
Best weeks for Movies:
week
1     316
44    243
40    215
9     207
26    195
Name: show_id, dtype: int64
```

In [115…

```
tvshows_week = (
    df_tvshows.groupby('week')['show_id']
    .count()
    .sort_values(ascending=False)
)
print("Best weeks for TV Shows:")
print(tvshows_week.head(5))
```

```
Best weeks for TV Shows:
week
27    85
31    79
24    75
35    73
13    73
Name: show_id, dtype: int64
```

In [116…

```
movies_month = (
    df_movies.groupby('month')['show_id']
    .count()
    .sort_values(ascending=False)
)
print("Best months for Movies:")
print(movies_month.head(5))
```

```
Best months for Movies:
month
7.0     565
4.0     550
12.0    547
1.0     546
10.0    545
Name: show_id, dtype: int64
```

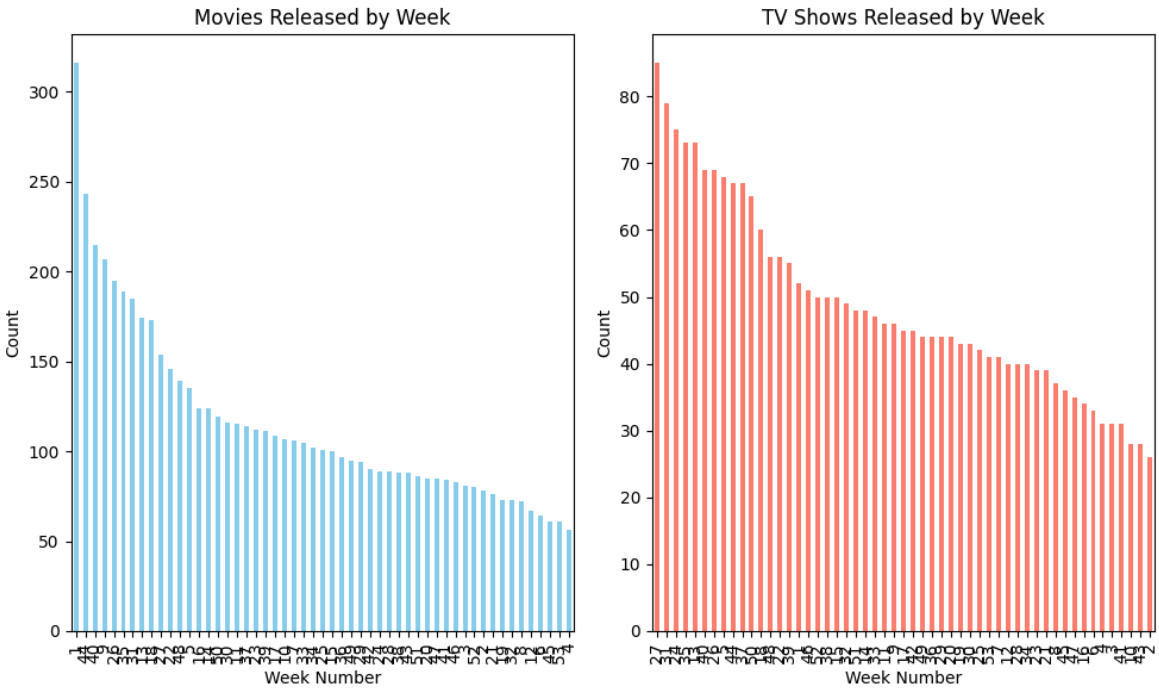
In [117…

```
tvshows_month = (
    df_tvshows.groupby('month')['show_id']
    .count()
    .sort_values(ascending=False)
)
print("Best months for TV Shows:")
print(tvshows_month.head(5))
```

```
Best months for TV Shows:
month
7.0     254
12.0    250
9.0     246
6.0     232
8.0     230
Name: show_id, dtype: int64
```

In [120…

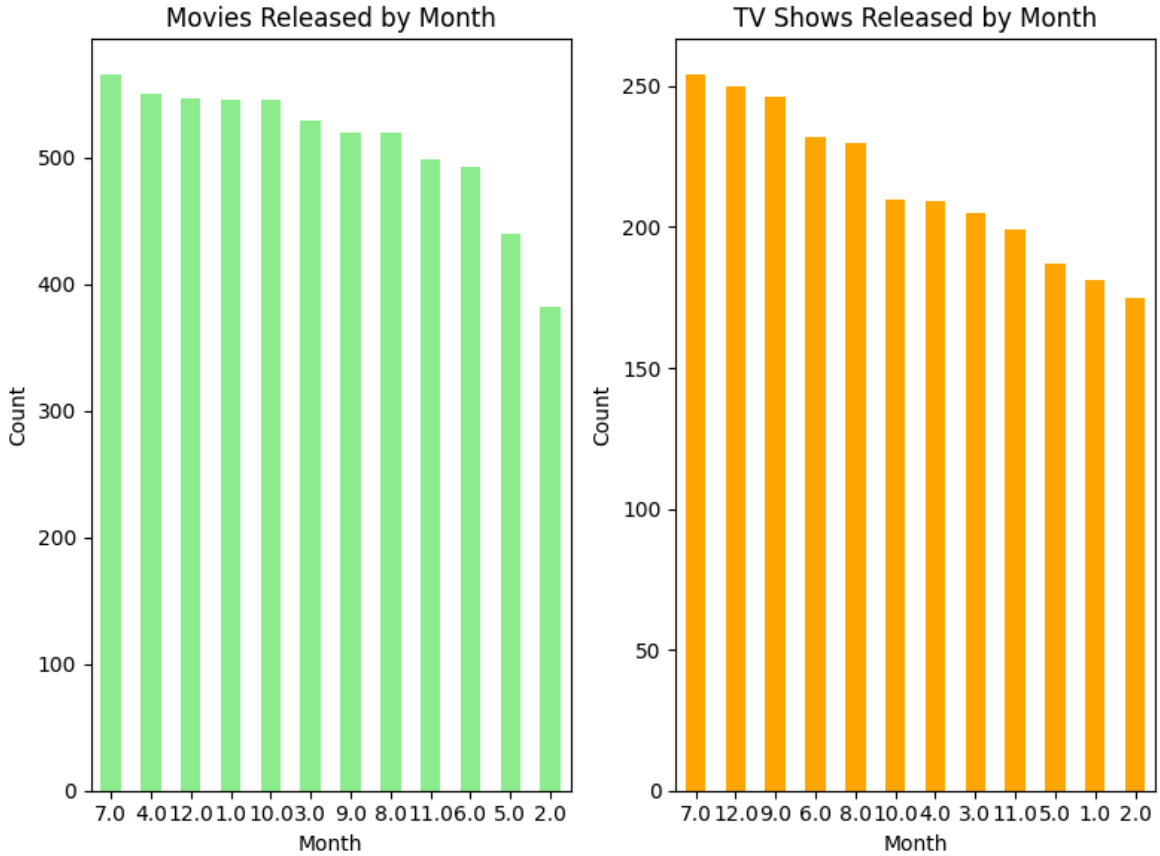
```
plt.figure(figsize=(10, 6))

# Movies by week
plt.subplot(1, 2, 1)
movies_week.plot(kind='bar', color='skyblue')
plt.title('Movies Released by Week')
plt.xlabel('Week Number')
plt.ylabel('Count')
plt.xticks(rotation=90)

# TV Shows by week
plt.subplot(1, 2, 2)
tvshows_week.plot(kind='bar', color='salmon')
plt.title('TV Shows Released by Week')
plt.xlabel('Week Number')
plt.ylabel('Count')
plt.xticks(rotation=90)

plt.tight_layout()
plt.show()
```



In [162…

```
plt.figure(figsize=(8, 6))

# Movies by month
plt.subplot(1, 2, 1)
movies_month.plot(kind='bar', color='lightgreen')
plt.title('Movies Released by Month')
plt.xlabel('Month')
plt.ylabel('Count')
plt.xticks(rotation=0)

# TV Shows by month
plt.subplot(1, 2, 2)
tvshows_month.plot(kind='bar', color='orange')
plt.title('TV Shows Released by Month')
plt.xlabel('Month')
plt.ylabel('Count')
plt.xticks(rotation=0)
```

```
plt.tight_layout()
plt.show()
```



top 10 actors who have appeared in most movies or TV shows.

In [104…

```
df_cast = df.copy()
df_cast['cast'] = df_cast['cast'].astype(str).str.split(', ')
df_cast = df_cast.explode('cast')
df_cast = df_cast.drop(df_cast[df_cast['cast'] == 'Unknown Cast'].index,
```

In [186…

```
top_actors = (
    df_cast.groupby('cast')['title']
    .nunique()  # count unique shows/movies per actor
    .sort_values(ascending=False)
    .head(5)
)

print("Top 5 Actors by number of shows/movies:")
print(top_actors)
```

```
Top 5 Actors by number of shows/movies:
cast
Anupam Kher         43
Shah Rukh Khan      35
Julie Tejwani       33
Naseeruddin Shah    32
Takahiro Sakurai    32
Name: title, dtype: int64
```

In [185…

```
top_actors_movies = (
    df_cast[df_cast['type']=='Movie'].groupby('cast')['title']
    .nunique()
    .sort_values(ascending=False)
    .head(5)
)
print("\nTop 5 Actors in Movies:")
print(top_actors_movies)

# Top actors in TV Shows
top_actors_tv = (
    df_cast[df_cast['type']=='TV Show'].groupby('cast')['title']
    .nunique()
    .sort_values(ascending=False)
    .head(5)
)
print("\nTop 5 Actors in TV Shows:")
print(top_actors_tv)
```

```
Top 5 Actors in Movies:
cast
Anupam Kher         42
Shah Rukh Khan      35
Naseeruddin Shah    32
Om Puri             30
Akshay Kumar        30
Name: title, dtype: int64

Top 5 Actors in TV Shows:
cast
Takahiro Sakurai    25
Yuki Kaji           19
Ai Kayano           17
Junichi Suwabe      17
Daisuke Ono         17
Name: title, dtype: int64
```

top 10 directors who have appeared in most movies or TV shows.

In [182…

```
df_director = df.copy()
df_director['director'] = df_director['director'].astype(str).str.split('
df_director = df_director.explode('director')
df_director = df_director.drop(df_director[df_director['director'] == 'Un
```

In [183…

```
top_directors = (
    df_director.groupby('director')['title']
    .nunique()
    .sort_values(ascending=False)
    .head(5)
)
print("Top 5 Directors by number of shows/movies:")
print(top_directors)
```

```
Top 5 Directors by number of shows/movies:
director
nan              2634
Rajiv Chilaka      22
Jan Suter          21
Raúl Campos        19
Marcus Raboy       16
Name: title, dtype: int64
```

In [184…

```
top_directors_movies = (
    df_director[df_clean['type']=='Movie'].groupby('director')['title']
    .nunique()
    .sort_values(ascending=False)
    .head(5)
)
print("\nTop 5 Directors in Movies:")
print(top_directors_movies)

# Directors for TV Shows
top_directors_tv = (
    df_director[df_clean['type']=='TV Show'].groupby('director')['title']
    .nunique()
    .sort_values(ascending=False)
    .head(5)
)
print("\nTop 10 Directors in TV Shows:")
print(top_directors_tv)
```

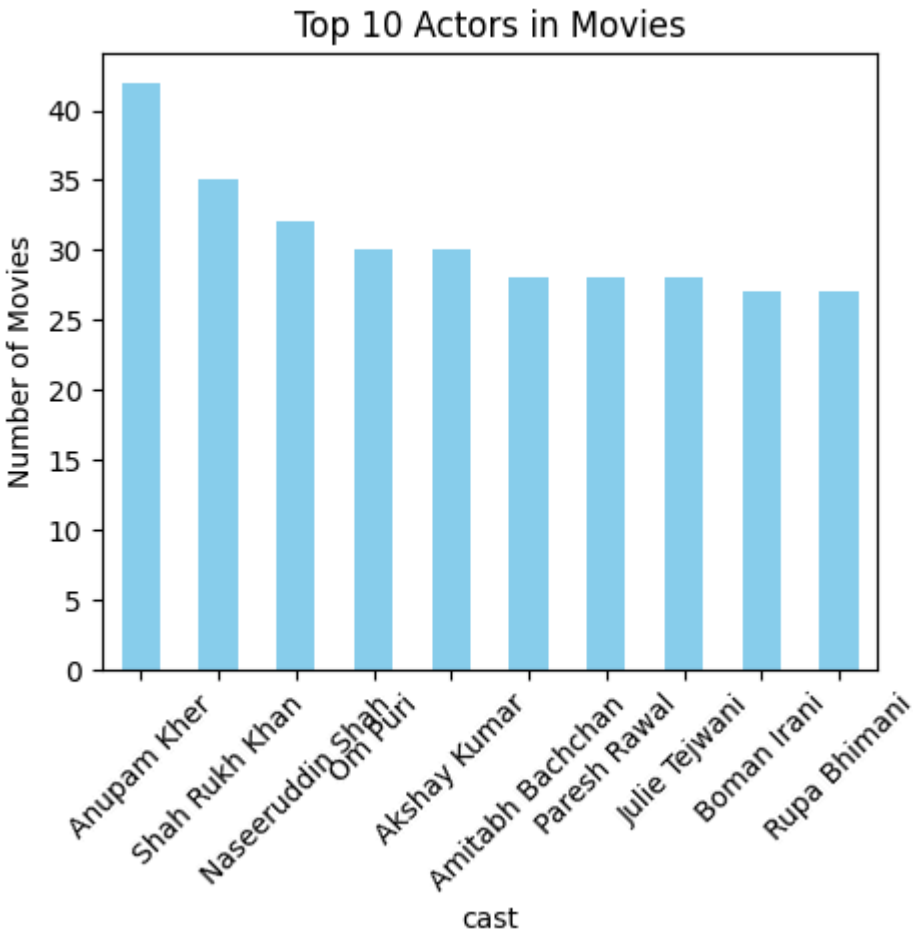
```
Top 5 Directors in Movies:
director
nan                    1944
Raúl Campos              15
Jan Suter                15
Cathy Garcia-Molina      13
Jay Karas                12
Name: title, dtype: int64

Top 10 Directors in TV Shows:
director
nan                  690
Rajiv Chilaka         19
Toshiya Shinohara      7
Youssef Chahine        7
Steven Spielberg       6
Name: title, dtype: int64
```

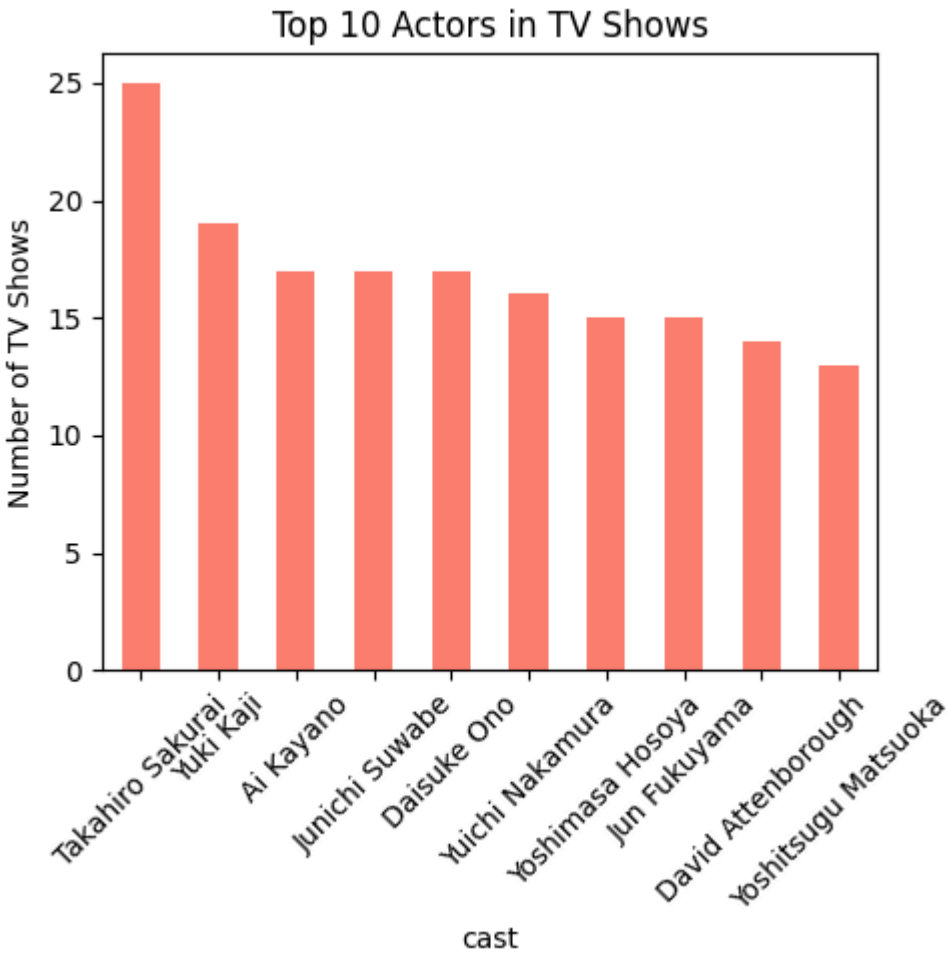
```
/tmp/ipython-input-834759248.py:2: UserWarning: Boolean Series key will be
reindexed to match DataFrame index.
  df_director[df_clean['type']=='Movie'].groupby('director')['title']
/tmp/ipython-input-834759248.py:12: UserWarning: Boolean Series key will b
e reindexed to match DataFrame index.
  df_director[df_clean['type']=='TV Show'].groupby('director')['title']
```

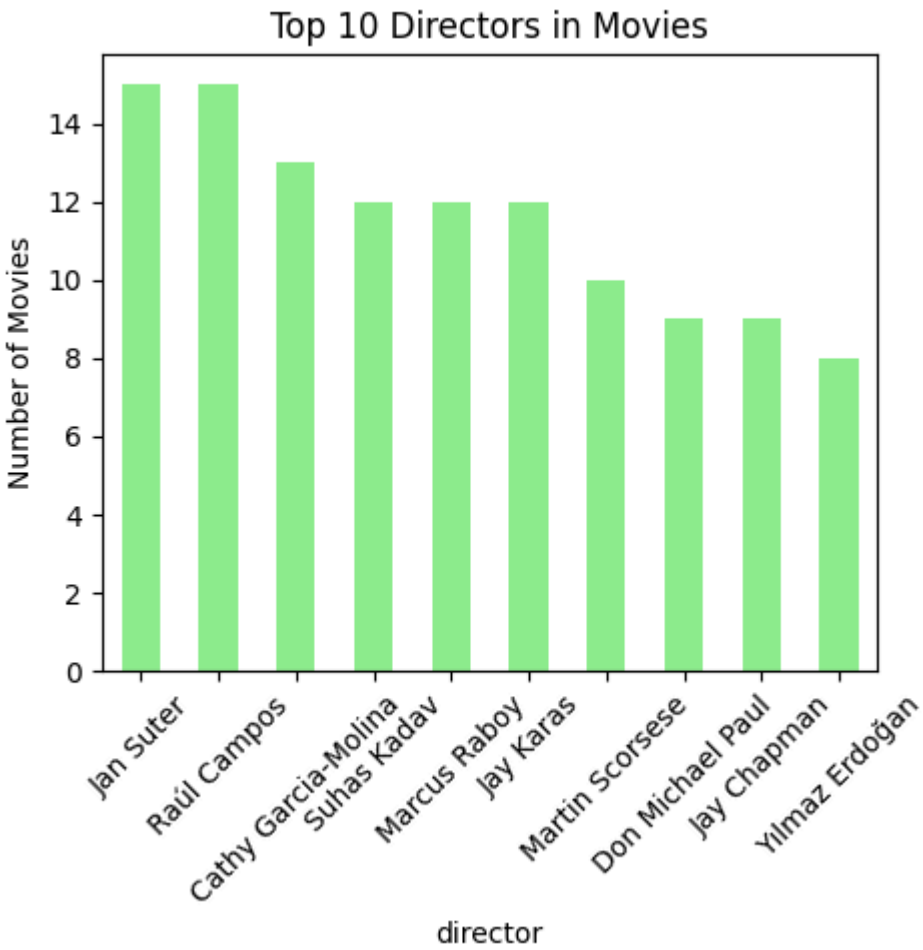
In [109…

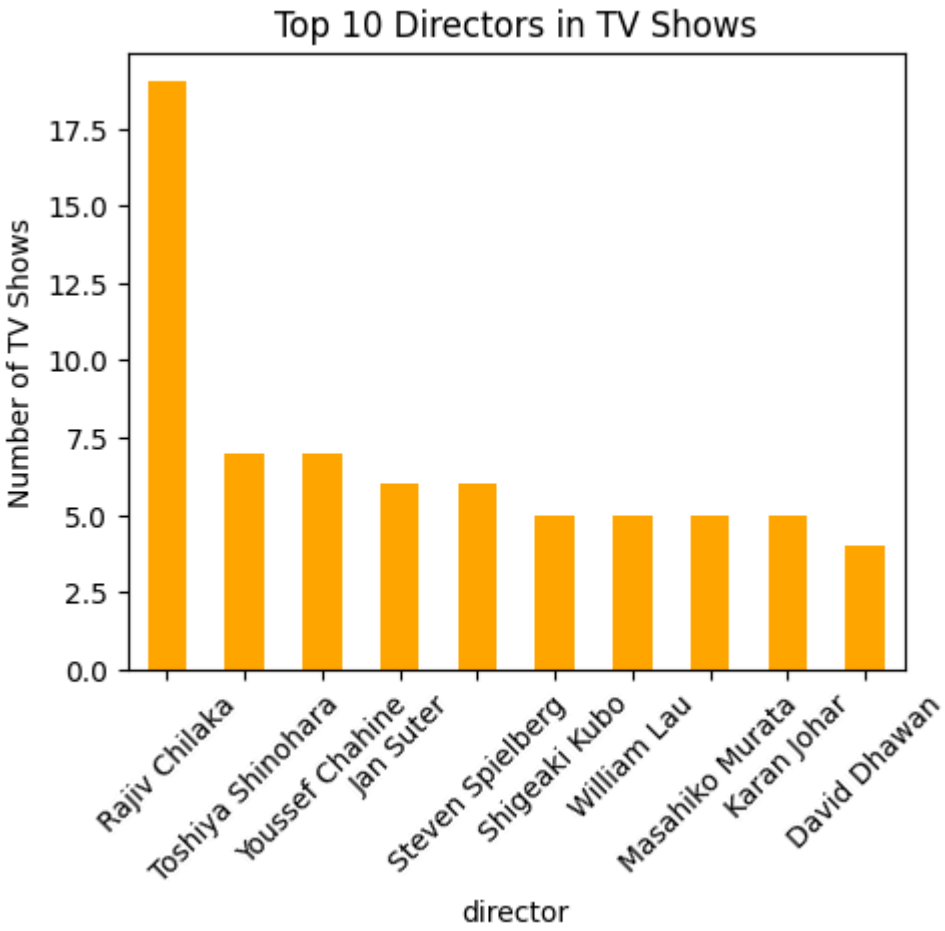
```
# Actors in Movies
plt.figure(figsize=(5,4))
top_actors_movies.plot(kind='bar', color='skyblue')
plt.title("Top 10 Actors in Movies")
plt.ylabel("Number of Movies")
plt.xticks(rotation=45)
plt.show()
```

```
# Actors in TV Shows
plt.figure(figsize=(5,4))
top_actors_tv.plot(kind='bar', color='salmon')
plt.title("Top 10 Actors in TV Shows")
plt.ylabel("Number of TV Shows")
plt.xticks(rotation=45)
plt.show()

# Directors in Movies
plt.figure(figsize=(5,4))
top_directors_movies.plot(kind='bar', color='lightgreen')
plt.title("Top 10 Directors in Movies")
plt.ylabel("Number of Movies")
plt.xticks(rotation=45)
plt.show()

# Directors in TV Shows
plt.figure(figsize=(5,4))
top_directors_tv.plot(kind='bar', color='orange')
plt.title("Top 10 Directors in TV Shows")
plt.ylabel("Number of TV Shows")
plt.xticks(rotation=45)
plt.show()
```




Top 10 Actors in TV Shows
Number of TV Shows
0
5
10
15
20
25
Takahiro Sakurai
Yuki Kaji
Ai Kayano
Junichi Suwabe
Daisuke Ono
Yuichi Nakamura
Yoshimasa Hosoya
Jun Fukuyama
David Attenborough
Yoshitsugu Matsuoka
cast


Top 10 Directors in Movies
Number of Movies
0
2
4
6
8
10
12
14
Jan Suter
Raúl Campos
Cathy Garcia-Molina
Suhas Kadav
Marcus Raboy
Jay Karas
Martin Scorsese
Don Michael Paul
Jay Chapman
Yılmaz Erdoğan
director



genre which are more popular or produced more for movies

In [95]:
```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

df_movies = df[df['type'] == 'Movie'].copy()

# Split and explode genres
df_movies['listed_in'] = df_movies['listed_in'].astype(str).str.split(',
df_movies = df_movies.explode('listed_in')

# Clean extra spaces
df_movies['listed_in'] = df_movies['listed_in'].str.strip()
```

In [97]:
```
# Join all genres into a single string
genre_text = " ".join(df_movies['listed_in'].dropna().tolist())

# Generate the Word Cloud
wordcloud = WordCloud(width=800, height=400, background_color='white', co

# Plot the Word Cloud
plt.figure(figsize=(8,7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Most Produced/Popular Genres")
plt.show()
```

Number of days it takes for the movie to add to Netflix after the release of the movie

In [153…

```
# Ensure date_added is datetime
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')

# Convert release_year to a datetime (assuming Jan 1 of that year)
df['release_date'] = pd.to_datetime(df['release_year'].astype(str) + '-01
```

In [154…

```
df_movies = df[df['type'] == 'Movie'].copy()
```

In [155…

```
# Difference in days between date_added and release_date
df_movies['days_to_netflix'] = (df_movies['date_added'] - df_movies['rele
df_movies.head(2)
```

Out[155…

	show_id	type	title	director	cast	country	date_added	release_y
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	Unknown Cast	United States	2021-09-25	2
6	s7	Movie	My Little Pony: A New Generation	Robert Cullen, José Luis Ucha	Vanessa Hudgens, Kimiko Glenn, James Marsden, …	Unknown Country	2021-09-24	2

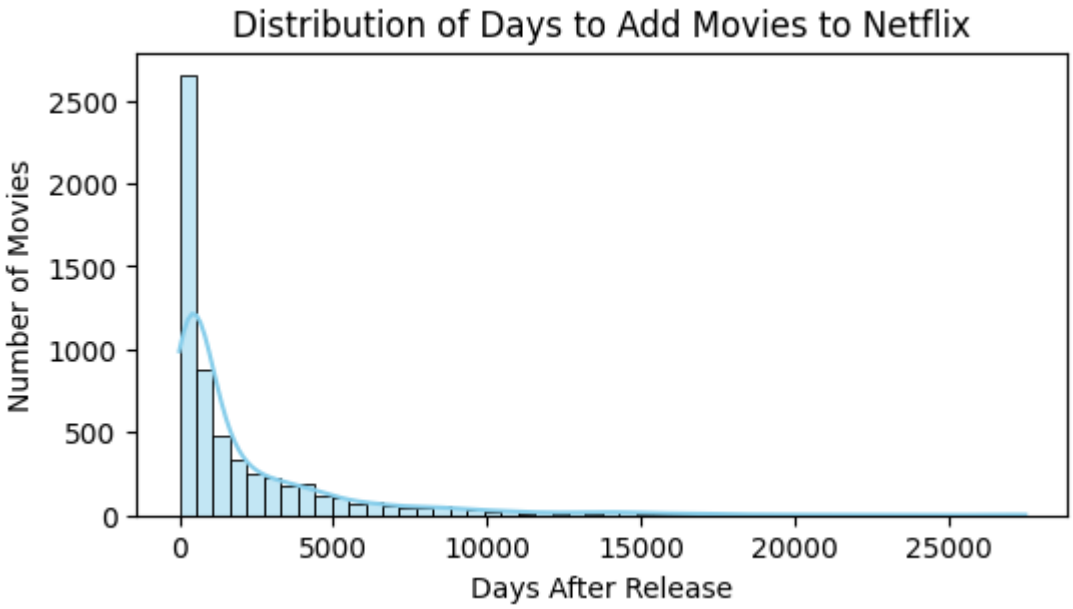
In [156…

```
df_movies_clean = df_movies[df_movies['days_to_netflix'] >= 0]

# Mode
mode_days = df_movies_clean['days_to_netflix'].mode()[0]
print(f"The most common number of days to add a movie to Netflix is: {mod
```

```
The most common number of days to add a movie to Netflix is: 334 days
```

In [157…

```
plt.figure(figsize=(6,3))
sns.histplot(df_movies_clean['days_to_netflix'], bins=50, kde=True, color
plt.title("Distribution of Days to Add Movies to Netflix")
plt.xlabel("Days After Release")
plt.ylabel("Number of Movies")
plt.show()
```



Insights

Comments on the Range of Attributes

Dataset spans 1925 – 2021 release years, showing both classics and recent productions.

Duration:

Movies: range from a few minutes to over 200 minutes.

TV Shows: range from 1 season to 15+ seasons.

Countries: Content from 100+ countries, though US, India, UK, and Japan dominate.

Ratings: Cover family-friendly (TV-Y, PG) to mature (TV-MA, R), showing Netflix targets wide audiences.

Genres (listed_in): Rich variety including Drama, Comedy, Documentaries, International Content, Stand-up Specials, Anime.

Comments on the Distribution and Relationships

Type Distribution: Movies form ~70% of content; TV Shows ~30%. However, TV Shows surged after 2015.

Ratings: Majority of content is TV-MA and TV-14 → Netflix mainly targets teens and adults.

Genres: Drama, Comedy, International Shows, Documentaries dominate → most consumed globally.

Countries vs Genres:

US: Action, Comedy, Drama

India: Bollywood movies, Romances

Japan: Anime, TV Shows

Duration vs Type: Stand-up/Documentaries = shorter; Mainstream movies = 100–150 mins; TV shows vary widely.

Time Trends: Massive spike in new content from 2015 onwards → aligns with Netflix's global expansion.

Comments on Plots

Univariate Plots:

Genres: Drama & Comedy dominate → highlight audience demand.

Countries: US leads, India next → US still primary market but Asia growing.

Ratings: TV-MA most common → focus on mature audience.

Bivariate Plots:

Type vs Year: TV shows grew rapidly post-2015.

Rating vs Type: TV shows → mostly TV-MA; Movies spread across PG, R, TV-14.

Country vs Type: US = balanced; India = movie-heavy; Japan = TV-heavy (anime).

Genre vs Duration: Comedy, Documentary = short; Drama, Action = longer.

Boxplot Insights :

Drama and Action movies are generally longer, while Comedy and Documentaries are shorter.

Movies cluster around 90–120 minutes, but TV Shows vary widely (1–15+ seasons).

TV Shows surged after 2015, showing Netflix's pivot toward binge-worthy content.

Comedy and Drama remain consistent, while Documentaries and Action grew post-2010.

Genre influences both duration and release trends, shaping audience viewing habits.

Business Insights

Shift from Movies → TV Shows

Before 2015, movies dominated. After 2015, TV shows surged, showing Netflix's strategic shift to binge-worthy serial content.

Regional Growth Drivers

US still dominates, but India, South Korea, and Japan are rapidly growing contributors, especially in TV content.

Genre Popularity Trends

Drama, Comedy, and International shows are most produced → indicates steady audience preference.

Documentaries and Stand-up specials show rising demand for short, niche content.

Age Rating Patterns

Content skewed toward TV-MA & TV-14, showing Netflix caters mainly to teens and adults.

Kids content is underrepresented, which could be a gap to explore.

Duration Patterns

Movies average 90–120 mins.

Shorter movies/documentaries gaining traction (snackable content for mobile).

TV shows range widely (1–15 seasons), but most are 1–3 seasons → test-and-expand model.

Star Power Effect

Popular actors/directors (e.g., Shah Rukh Khan in India, Adam Sandler in the US) appear repeatedly → Netflix leverages recognizable faces for global reach.

Content Release Spike (2015–2020)

Aligns with Netflix's international expansion strategy, showing that more localized content helped rapid growth. boxplot business insights

Netflix shifted from movies to TV shows post-2015 to retain subscribers.

Drama, Comedy, and International content dominate audience preferences.

Mature-rated shows (TV-MA, TV-14) form the majority, while kids' content is limited.

Regional content (India, Korea, Japan) is gaining global traction, especially anime and K-dramas.

Documentaries, short specials, and multi-season shows drive engagement and retention.

Cross-Cultural Demand

Korean dramas, Japanese anime, and Indian Bollywood movies are consumed globally, not just locally.

Seasonal Release Patterns

Movies tend to release around holidays/festivals, TV shows around summer and winter breaks.

Niche Segments Emerging

Rise of True Crime Documentaries, Reality TV, and Mini-Series → reflects changing audience tastes.

Content Gaps

Kids/family-friendly shows are far fewer compared to adult content.

Limited content in African and Middle Eastern regions → growth opportunity.

Language Diversity

Non-English content (Spanish, Korean, Hindi, Japanese) is increasingly popular → English-only focus is outdated.

Retention Strategy

Multi-season shows (like Money Heist, Stranger Things) act as retention anchors, keeping subscribers engaged for years.

Experimentation with Formats

Netflix experiments with interactive shows (e.g., Bandersnatch), but volume is still low compared to demand.

Long-tail Content Value

Even older/less popular titles add value by filling niche audiences and increasing catalog depth.

Recommendations

Content Strategy

Increase TV Show Production → especially mini-series & multi-season thrillers to boost binge-watching and retention.

Invest in Global Content → double down on India, Korea, and Japan where international hits are being produced.

Diversify Genres → focus more on thrillers, true crime, reality shows, and anime as these are trending.

Expand Kids & Family Content → create more PG/TV-Y shows and movies to attract parents and young viewers.

Boost Documentary & Short-Format Content → useful for casual/mobile audiences.

Market Expansion

Local Partnerships → co-produce with regional studios in Africa, Middle East, and Latin America.

Promote Non-English Originals Globally → position K-dramas, Indian shows, and Anime as global products.

Localized Marketing → stronger advertising in India, South Korea, Japan, using local festivals, influencers, and language. Produce more binge-worthy TV shows, mini-series, and multi-season thrillers.

Expand family-friendly content and strengthen kids' catalog.

Invest in global originals (India, Korea, Japan) and market them worldwide.

Release movies during holidays and shows mid-year/year-end for maximum reach.

Partner with regional stars, add affordable plans, and use data to greenlight popular genres.

Customer Engagement

Seasonal Releases →

Movies: launch during festivals/holidays (Christmas, Diwali, Lunar New Year).

TV Shows: release mid-year & year-end for long-term retention.

Interactive & Gaming Content → expand experiments like Bandersnatch; explore tie-ins with gaming/AR.

Franchise Building → invest in creating multi-season franchises (like Stranger Things, Money Heist).

Retention & Value Creation

Star Collaborations → keep signing exclusive deals with A-list stars in US, India, and Korea.

Affordable Regional Pricing → introduce lower-cost, mobile-only plans in price-sensitive markets (India, Africa).

Content Recommendation Algorithms → highlight local + global hits side-by-side to encourage cross-cultural viewing.

Data-Driven Greenlighting → prioritize genres with proven global traction (thrillers, comedies, anime, true crime).